HCI - recap

1. HCI Foundations: Human, Computer and Interactions

Human-Computer interaction (HCI) is research in the design and the use of computer technology, which focuses on the **interfaces between people (users) and computers.**

HCI observes the ways humans interact with computers and **design** technologies that allow humans to interact with computers in novel ways.

Human: The human is the person who interacts with the computer system. Humans bring their own abilities, limitations, and expectations to the interaction. They use perception, memory, attention, and motor skills to understand the interface and perform actions. Designing good interfaces means taking into account how humans process information, how they make decisions, and how they physically interact with the system.

Computer: A computer is a system made up of different components, such as a screen, a keyboard, and a mouse or trackpad, along with other possible input and output devices. The type of devices used shapes the way users interact with the system. Different devices lead to different interfaces, which in turn support different interaction styles.

Interaction: Interaction is the communication between the user and the computer system.

<u>key concepts of interaction:</u>

- **domain**: the area of work under study (ex. graphic design)
- **goal**: what you want to achieve (ex. create a solid red triangle)
- **task**: the set of actions the user performs to reach the goal.

Donald Norman's Model of Interaction: Donald Norman focuses on the user's point of view during interaction. His model describes seven stages:

1. user establishes the goal (goal)
2. formulates intention – how to achieve the goal (execution)
3. the user identifies the actions needed at the interface. (execution)
4. executes action (execution)
5. the user observes what the system is showing (evaluation)
6. the user understands the system's feedback (evaluation)
7. the user evaluates if the result matches the goal (evaluation)

Problems arise if the actions the user plans do not match what the system allows, or if the system's feedback doesn't match the user's expectations.

Abowd and Beale Interaction Framework: this model extends Norman's by describing interaction as a translation between four components, each with its own "language":

1. user
2. input (how the user communicates to the system)

3. system (internal processing)
4. output (how the system communicates back to the user)

From user's mental model → to system input language & from system state → to output presented to the user.

Problems in interaction often arise when these translations are not smooth (e.g. unclear feedback, ambiguous commands, bad mapping).

Ergonomics: ergonomics studies the physical aspects of interaction — how the design fits the user's physical abilities and comfort.

Interaction styles: common ways users interact with systems include:

- Command line: typing instructions (mostly for experts).
- Natural language: speaking or typing using everyday language.
- Question/answer and forms: filling in data or making selections.
- Spreadsheets: data entry through tables and grids.
- **WIMP** (Windows, Icons, Menus, Pointers):
    - *Windows*: independent areas on the screen.
    - *Icons*: small images representing functions or files.
    - *Menus*: lists of actions or options.
        - Menu types: menu bar (pull-down), context menu (pop-up), cascading menus.
        - Design issues: wording, grouping, ordering, and shortcuts.
    - Point and click: selecting objects with minimal typing.
- 3D Interfaces: interaction in three-dimensional virtual spaces.

People only use a system if its perceived value is higher than its cost (in time, effort, or difficulty). To design good interactions: **make tasks easy and match the users' needs and expectations.**

---

2. User Requirements

Before designing a system, it's essential to understand both the product and its users. This phase helps identify what the system needs to do to satisfy real user needs.

Requirements gathering and analysis: A requirement is something the product must do or a quality that the product must have; users may not be aware of all requirements, may voice a perceived need but also forget some of them.

How are requirements gathered?

Analyzing Competitors: Start by researching similar products to understand:

- Strengths: what works well in existing products.
- Weaknesses: where they fail or frustrate users.
- Customer base: who uses them.

- Availability: how easily they can be accessed or purchased.
- Functionality & unique features: what makes each product special.
- Reputation: user reviews, ratings, and general public opinion.
- Requirements: what needs they aim to satisfy.

This analysis helps you avoid repeating mistakes and identify opportunities to improve.

Defining Users – Personas: A persona is a detailed, fictional profile representing a typical user of the system. It helps designers focus on real user needs instead of assumptions.

When creating personas, describe:

- Age
- Gender
- Education
- Occupation
- Technical skills
- Goals and motivations
- Frustrations
- Devices used
- Context of use

Although fictional, personas should feel realistic and be as detailed as possible to guide design decisions.

Scenarios and context of use: A scenario describes how a persona interacts with the system in a specific situation. It helps visualize how users will behave and what they expect.

Scenarios help explore:

- Interaction: what happens as the user operates the system.
  Cognition: what the user is thinking and expecting during interaction.
- Architecture: how the system responds internally.

Scenarios are useful to:

- Communicate ideas to designers, clients, and stakeholders.
- Validate other models by testing them against realistic situations.
- Show dynamics using both screenshots (appearance) and scenarios (behavior).

Methods for gathering user requirements: Once you have a general understanding of users, you need to gather real data. Possible methods include:

- **Interviews**: individual conversations to explore user needs deeply.
- **Surveys** and **questionnaires**: structured sets of questions used to collect data from a large number of users quickly. They help identify trends, preferences, and general opinions, but provide less depth than interviews.
- **Focus groups:** A small group of participants (usually 5-10 people) is brought together to discuss their experiences, needs, expectations, or reactions to a product,

design, or concept. A moderator leads the discussion, asks questions, and ensures everyone participates.

- **Field studies**: researchers observe users in their natural environment as they interact with systems or perform tasks. This method helps uncover real-world behaviors, challenges, and environmental factors that affect interaction.
- **Task analysis:** A detailed examination of how users perform tasks, breaking down each step to understand workflows, decision points, and potential difficulties. It helps identify where the system needs to provide support.
- **Card sorting:** Participants organize information into groups that make sense to them. This helps designers understand how users categorize and structure information, which informs the design of menus, navigation, and information architecture.
- **Group brainstorming and workshops:** Collaborative sessions where multiple stakeholders (users, designers, developers, etc.) generate ideas, discuss requirements, and identify problems or solutions together. Encourages creativity and shared understanding early in the design process.

Questionnaires: Questionnaires are a useful method to collect information quickly from users. They can be used both in the user requirements phase (to investigate user needs, expectations, preferences, priorities) and in the evaluation phase (to assess user satisfaction, attitudes, opinions, relevance of services, or to track trends over time).

How to create a questionnaire?

1. Define your goal: decide exactly what information you want to collect.
2. Identify your audience: determine who should fill out the questionnaire.
3. Choose the data collection method: online form, paper, interviews, etc.
4. Draft the questionnaire: carefully write the questions, paying attention to wording, content, format, structure, and layout.
5. Pilot the questionnaire: test it with colleagues or a small sample of the target audience and revise if needed.
6. Plan timing: decide when and for how long the questionnaire will be open.
7. Collect and analyze the responses.
8. Report and use the findings to inform design or evaluation decisions.

What makes a good survey?

- ➔ Questions are **clear, specific, and precise**.
- ➔ The set of questions allows for **detailed and meaningful answers**.
- ➔ Predefined answer options (if any) are well-constructed and appropriate.
- ➔ **Optional open fields** allow users to add extra information if necessary.
- ➔ Unnecessary or irrelevant questions are avoided.
- ➔ Use **conditional logic (skip patterns)** to avoid asking non-relevant questions to some users.
- ➔ Keep both questions and answers **short, simple, and easy to read**.
- ➔ Avoid using different or confusing scales (e.g. mixing 1-5 scales with 1-10 scales).
- ➔ Ensure the format supports easy and comfortable answering.

3. Designing Mobile Interfaces

The goal of HCI is to enable effective **operation and control of technology by the user**, while the system provides feedback that supports decision-making.
The objective is to bridge the gap between humans and technology, making digital systems more accessible, efficient, and satisfying to use.

User Interface (UI): The User Interface is the point where the user interacts with a system, software, or device. It allows the user to control the technology and receive feedback from it.

Types of User Interfaces:

- **Graphical User Interface (GUI):** Uses graphical elements such as windows, icons, menus, and a pointer for navigation and interaction – *GUI = what you see on the screen;*
- **Touch User Interface (TUI):** Allows users to interact directly with the device via the touchscreen, using gestures such as taps, swipes, and pinches – *TUI = how you interact with it.*

The Mobile Revolution: The success of mobile devices is due to their portability, intuitive interfaces, and continuous technological advancements. However, mobile interface design brings unique challenges that require dedicated, platform-specific solutions.

Some of these challenges include:

- Limited screen space: Interfaces must adapt to small screens without overwhelming the user;
- Diverse screen sizes and resolutions: The design should be responsive and work across different devices;
- Changing usage environments: Lighting, noise, and physical conditions may affect usability;
- Touch interaction limitations: Accuracy can be difficult on small screens, requiring careful layout and larger interactive elements;
- Typing difficulty: Text input is more challenging on touchscreens;
- Limited user attention: Mobile users often interact briefly and in distracting environments, requiring fast and simple interactions.

Designing Usable Mobile Interfaces: Designing mobile interfaces requires applying principles that make apps easy to use, effective, and user-centered.

There are **three key ingredients** to follow:

1. **Use guidelines and mobile patterns**
2. **Prioritize the initial user experience:**
    a. the first impression of a mobile interface is critical
    b. even with excellent design, code, and a backend, a confusing interface leads to user failure
3. **Use prototyping**
    a. employ advanced tools for mobile interface mockups and interaction design

Use guidelines and mobile patterns: One common approach is User-Centered Design (**UCD**), which prioritizes the needs, goals, and limitations of users at every stage of design.

There are also **established rules** and **collections** like:

Shneiderman's 8 Golden Rules

1. Strive for consistency: keep interface elements and behaviors uniform.
2. Enable frequent users to use shortcuts: allow power users to act quickly.
3. Offer informative feedback: show users what the system is doing.
4. Design dialogues to yield closure: make it clear when a task is complete.
5. Offer error prevention and simple error handling: minimize errors and allow easy recovery.
6. Permit easy reversal of actions: allow undo to keep users confident.
7. Keep users in control: avoid overwhelming users; ensure they feel in charge.
8. Reduce short-term memory load: display information visibly to reduce memory effort.

Norman's 7 Principles

1. Use both *knowledge in the world* and *knowledge in the head*.
2. *Simplify* task structures to reduce memory load.
3. Make things *visible*.
4. Ensure proper mapping between actions and effects.
5. Use constraints (natural or artificial) to guide users.
6. Design for *error* (anticipate and handle user mistakes).
7. *Standardize* when possible.

The Pillars of Usable Mobile Design:

1. **Information Architecture**: Defines how users navigate and access information.

- The **site map** is used to organize content and define navigation paths.
- Limit navigation depth (ideally no more than 5 navigation levels).
- Always include a **back button** to prevent users from getting lost.

2. **Interface Design:** Focuses on the visual structure of the app.
   Main layout: header, navigation, content area, footer.

- Key design principles:
  - Axis & symmetry: create balanced, visually pleasing layouts.
  - Hierarchy: more important elements are visually larger.
  - Rhythm: consistency in repeated elements.
  - Breaks: intentional interruptions to highlight key information.
- A major challenge is scalability: adapting to different screen sizes while maintaining clarity.
- Avoid overwhelming users when many tasks or actions are presented.

3. **Interaction Design:** Defines how users interact with the app, either:
   - Directly: through clear feedback and control during task performance.

    - Indirectly: through sensors or system intelligence.
- Design for thumb-friendly areas (reachable zones).
- Minimize typing; gather information automatically from the OS when possible.

4. **Information Design:** Focuses on how content is visually displayed.
   Key aspects:
- Design for fat fingers (lol): use large, easily tappable buttons.
- Design for distracted users: keep interfaces simple and easy to use.
- Use proper colors, typography, and motion to support readability and **clarity.**

---

4. User Centered Design (UCD)

User-Centered Design is an approach (a pattern, as we already mentioned) where design decisions are based on information gathered about the users. The goal is to create usable systems by involving potential users throughout the design process.

UCD is iterative: users are involved during all phases of the system's life cycle to ensure the product meets usability goals.

Definition of usability: Usability describes how easy and effective it is to use a system. Formally, it's:

"The extent to which a product can be used by specified users to achieve specified goals with effectiveness, efficiency, and satisfaction in a specified context of use."

Usability is defined by 5 main quality components (L-E-M-E-S):

1. **Learnability**: how easy it is for new users to accomplish basic tasks;
2. **Efficiency**: how quickly users can perform tasks after learning the system.
3. **Memorability**: how easily users can return to using the system after not using it for a while.
4. **Errors**: how many errors users make, how severe they are, and how easily users can recover.
5. **Satisfaction**: how pleasant the system is to use.

Usability measures

- Effectiveness: how accurately and completely users achieve their goals.
- Efficiency: how quickly and easily users can perform their tasks.
- Satisfaction: user comfort and positive attitude towards the system.

Characteristics of UCD

- Know your users.
- Actively involve users early and throughout the process.
- Use rapid and frequent iterations with usability testing.
- Work with a multidisciplinary team (designers, developers, usability experts, etc.)

5. Interaction Design Basics

In Interaction Design, we are designing not just how the interface looks, but **how people interact with the system** to achieve their goals. The focus is on creating meaningful, smooth, and effective interactions between the user and the system.

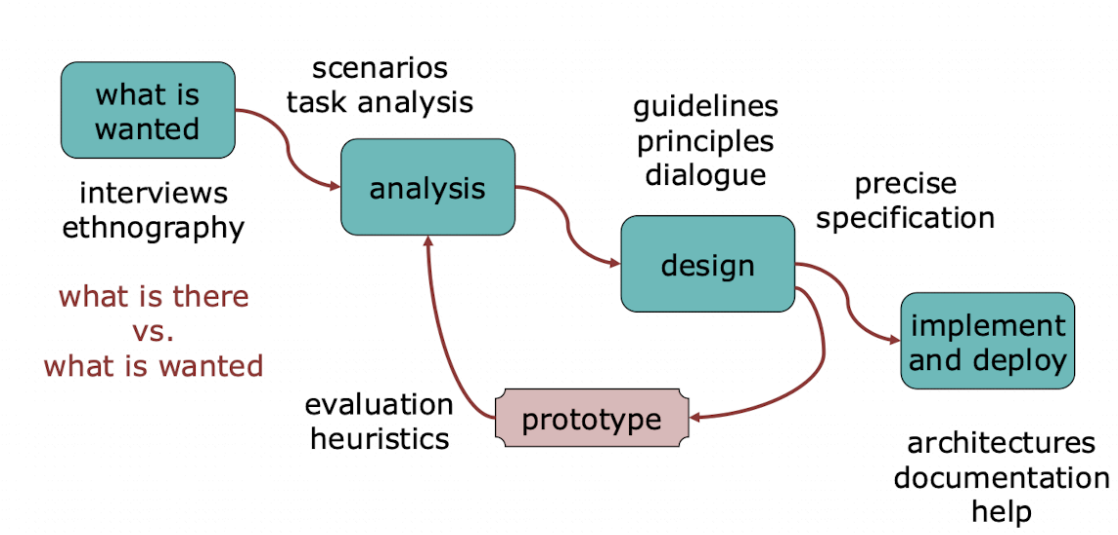
What is design? **Design** means achieving **goals** while managing **constraints**.

- **goals** → the purpose of the system (what the user wants to achieve).
- **constraints** → limitations of technology, platforms, materials, and time.
- **trade-offs** → decisions made to balance conflicting demands (e.g. speed vs. features).

The golden rule of design is **"*understand your materials*."**

In HCI, our materials include: the system itself (hardware & software), the people who interact with it and the context of use.

The design process typically includes:



1. <u>Requirements gathering (What is wanted):</u>
   - Interviews, observations, user studies.
2. <u>Analysis:</u>
   - Scenarios, task analysis, personas.
3. <u>Design:</u>
   - Apply guidelines, principles, and create dialogue structures.
4. <u>Prototype:</u>
   - Build early models of the interface.
   - Evaluate using heuristics, user testing, or technology probes.
5. <u>Iterative Analysis:</u>
   - Analyze feedback and refine the design.
6. <u>Implementation and Deployment:</u>

- Build the system with full specifications and documentation.
- Provide user support (help files, training, onboarding).

Because time and resources are limited, **trade-offs** are often necessary — not every feature or design element can be perfect.

Four golden rules for supporting user navigation: A good interaction design helps users always know:

1. Where they are: Use breadcrumbs or navigation indicators to show their current location.
2. What they can do: Make all available actions visible and clear.
3. Where they are going (or what will happen): Indicate what to expect when an action is taken.
4. Where they have been (or what they have done): Provide history or feedback of completed actions.

---

6. Cognitive Models

Cognitive models aim to describe how users think, understand, plan, and act when interacting with a system. They help designers better understand the user's mental processes during interaction, such as:

- **Understanding:** how users make sense of the interface.
- **Knowledge:** what users know or assume.
- **Intentions:** what users want to achieve.
- **Processing:** how users plan actions, make decisions, and execute tasks.

These models are used to predict user behavior, analyze errors, and improve interface design by aligning it with how people naturally think and act.

Goals vs Tasks:

- **Goals:** the intentions or outcomes the user wants to achieve (what they would like to be true).
- **Tasks:** the actions the user performs to achieve the goals.

Most used cognitive models: GOMS, HTA and CCT

GOMS (Goals, Operators, Methods, Selection Rules): Models how users achieve their goals by breaking tasks into small steps (operators), different ways to perform tasks (methods), and rules for choosing between them (selection). It helps predict task time and complexity.

GOMS is based on an **information processing framework**, assuming separate stages for:

```
GOAL: CLOSE-WINDOW
.   [select GOAL: USE-MENU-METHOD
            .   MOVE-MOUSE-TO-FILE-MENU
            .   PULL-DOWN-FILE-MENU
            .   CLICK-OVER-CLOSE-OPTION
           GOAL: USE-CTRL-W-METHOD
            .   PRESS-CONTROL-W-KEYS]

For a particular user:

   Rule 1: Select USE-MENU-METHOD unless another
           rule applies
   Rule 2: If the application is GAME,
           select CTRL-W-METHOD
```

- Perception
- Cognition
- Motor processing
- Memory (short-term, long-term, and working memory)

HTA (Hierarchical Task Analysis): Breaks down tasks into subtasks in a hierarchical structure to analyze how users complete goals step-by-step. Focuses on actions and task organization rather than internal thinking/cognitive processes. It is useful for understanding how people physically and sequentially perform tasks to achieve goals.

CCT (Cognitive Complexity Theory): Models how difficult it is for users to learn a system by analyzing the complexity of the rules they must learn (if-then rules) and how these rules match the system's behavior. Users' actions are modeled using two parallel descriptions:

1. **Production rules:** if-condition-then-action rules describing user behavior.
2. **Device models:** generalized transition networks representing system behavior.

- Production rules are in long-term memory
- Model working memory as attribute-value mapping:

  (GOAL perform unit task)
  (TEXT task is insert space)
  (TEXT task is at 5 23)
  (CURSOR 8 7)
- Rules are pattern-matched to working memory,

  e.g., LOOK-TEXT task is at %LINE %COLUMN is true, with LINE = 5 COLUMN = 23.

Dialogue notation models (formal representations): TAG & BNF – Dialogue notations are **formal ways to describe how interaction between the user and the system happens step-by-step**. They focus on the structure and rules of the "conversation" between user and system — which actions are allowed, in which order, and how the system responds.

Since interaction can be seen like a "language" (user gives input, system gives output), these notations often borrow concepts from formal language theory, hence the name *linguistic notations*.

They help designers:

- Specify valid sequences of actions.
- Detect inconsistencies or missing actions.
- Clearly describe interaction flows before implementation.

Task Action Grammar (TAG): It's a grammar-based method used to describe and structure user actions in a system. It focuses on making interactions consistent by using grammar rules that represent how users understand tasks. TAG includes the user's knowledge in its rules and adds extra meaning to actions, helping designers formalize and organize how actions are performed.

Backus-Naur Form (BNF): Backus-Naur Form (BNF) is a notation from computer science that describes the structure of dialogues based purely on syntax. It is Purely syntactic: defines valid action sequences (like writing a grammar for a programming language).

Model/Notation	Type	Focus
BNF	Dialogue Notation	Syntax of valid interaction sequences
TAG	Dialogue Notation	Structure + semantics of user actions
GOMS	Cognitive Model	User goals, methods, actions
HTA	Cognitive Model	Task decomposition and plans
CCT	Cognitive Model	Learning complexity (rules & system behavior)

Limitations of cognitive models:
Many cognitive models focus on reasoning, goals, and tasks but often ignore aspects like:

- User observation and perception;
- Emotional states or real-world variability.

They can also be:

- Post hoc (applied after design, rather than during).
- Less suitable for novice users (since they often assume expert-level behavior and knowledge).
- Limited in capturing full cognitive complexity of real user

---

7. Hierarchical Task Analysis (HTA)

HTA is a cognitive method used to represent and organize the tasks users perform to achieve specific goals. It helps designers analyze how users interact with a system and how interfaces should be designed to support these interactions.

HTA in the context of Concept Modeling: Conceptual modeling means describing aspects of the real world for better understanding and communication. Different domains use different models (e.g., E-R diagrams for databases, UML for software design, etc.).

In HCI, conceptual models help us:

- Structure and analyze user tasks.
- Represent how users interact with systems.
- Define what the interface should support.

Task Analysis: Task analysis is the general process of studying users' tasks in detail to improve system design.

- Describes tasks and subtasks at a detailed level.
- Helps identify the information and features the system needs to support.
- Answers questions like:
    - What are the users trying to do?
    - What the user knows about the task and how is it organised?
    - How do they do it?

- What steps are involved?

The right approach is to make task decomposition as user friendly as possible!

General method: observe → collect unstructured lists of words and actions → organize using notation or diagrams.

HTA is a **specific type of task analysis** where tasks are broken down into smaller subtasks in a hierarchical structure.

Key concepts of HTA:

- Goals: what the user wants to achieve.
- Tasks: actions performed to achieve the goal.
- Subtasks: smaller steps that compose a task.
- Plans: describe the order and logic of executing tasks/subtasks.

HTA uses **graphical notations** (diagrams, trees, hierarchies) to visually represent how tasks are structured and organized.

General method for performing HTA:

1. Observe users: watch how they perform tasks.
2. Collect lists of actions: unstructured lists of everything the user does.
3. Organize the actions: group related tasks and define higher-level tasks.
4. Decompose tasks: break tasks down into subtasks until reaching the most basic actions.
5. Create the HTA diagram: represent the full structure visually.

Generating the Hierarchy in HTA:

1. Start by collecting a list of tasks, using information from user research and competitor analysis.
2. Focus on tasks that are relevant to user goals; expand only the most important ones.
3. Group related tasks into higher-level tasks.
4. Decompose tasks into smaller subtasks until reaching the lowest useful level.

Stopping Rules: When do we stop decomposing tasks?

- Stop when the task description is simple enough to be clear (e.g., "empty dust bag" may be sufficient).
- Only expand relevant tasks that are important for system design.
- Motor actions (physical interactions) typically represent the lowest sensible level of decomposition.

Heuristics to check and improve HTA: To review or refine the hierarchy (whether written as text or diagram), you can apply these heuristics:

- **Paired actions:** ensure actions that logically belong together are kept together (e.g., open → close, start → stop).

- **Restructure:** reorganize tasks for better clarity or logical flow.
- **Balance:** check that similar tasks are described at a similar level of detail.
- **Generalize:** abstract common tasks into higher-level general categories when possible.

Types of plan: Plans describe **how and when** tasks and subtasks are executed.

fixed sequence	- 1.1 then 1.2 then 1.3
optional tasks	- if the pot is full 2
wait for events	- when kettle boils 1.4
cycles	- do 5.1 5.2 while there are still empty cups
time-sharing	- do 1; at the same time ...
discretionary	- do any of 3.1, 3.2 or 3.3 in any order
mixtures	- most plans involve several of the above

Waiting…: Some actions like **waiting** can be either tasks or part of a plan: If waiting requires attention or activity, treat it as a **task** (e.g., monitoring a timer). If waiting is passive and only marks when the next event happens, treat it as part of the **plan**.

(nelle slides "Task Models" ci stanno altre cose che ho deciso di skippare perchè idgaf)

---

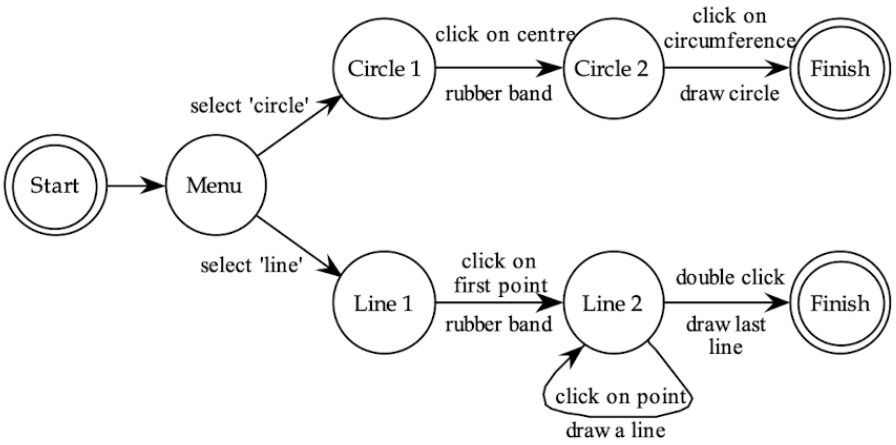
8. Dialogue notations and design

Dialogue notation and design help describe, visualize, and plan the step-by-step interaction between users and systems, ensuring smooth, usable interaction flows.

Dialogue: a dialogue is the conversation between two or more parties – in user interfaces, it refers to the structure of the interaction. HCI dialogue is very constrained because HCI both the user and system can only interact through limited, well-defined actions and responses that the system can recognize and handle correctly.

Graphical notations: graphical notations are used to visually represent the flow of interaction between the user and the system and they help designers map out how the system responds to user actions and how different states connect.

State Transition networks (STN):



- In the STN notation the interaction is modeled as a set of **states** (represented by circles) and **actions or events** (represented by arrows between the states).
- When the user performs an action or an event occurs, the system moves from one state to another.

Characteristics:

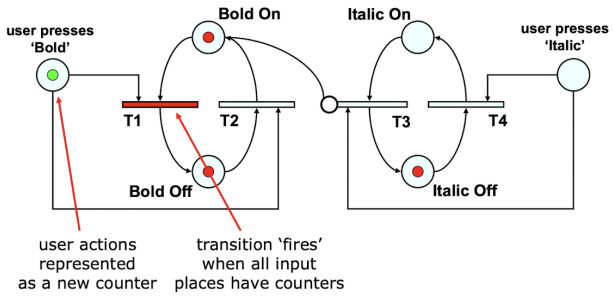
- The diagrams tend to be **state-heavy** → most of the complexity lies in naming and managing the states.
- The **labels inside states** often hold little detailed information because states are sometimes hard to describe precisely.
- **Transitions (arcs)** carry most of the interaction details (since they describe what the user/system actually does).

pros	cons
- Easier to visualize how users move through the system - Can model both simple and complex systems.	- For very complex systems, STNs can become crowded and hard to manage. - The more detailed the interaction, the heavier the diagram becomes.

Hierarchical STNs: Complex systems can be broken into multiple sub-STNs to manage complexity (each sub-STN represents a part of the system).

Petri Nets

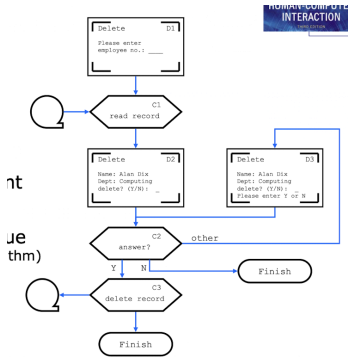
- Similar to STNs but can model parallel processes and synchronization.



- Useful when multiple events or actions happen at the same time or depend on one another.
- Use circles (places), bars (transitions), and tokens to represent system state changes.

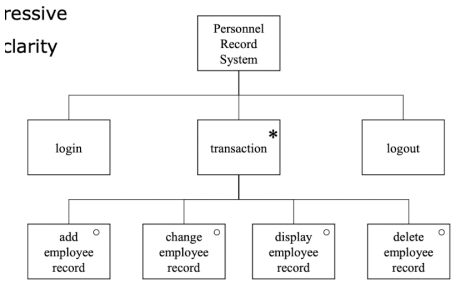
Flowcharts

- A classic way to represent processes using:
  - Rectangles for actions
  - Diamonds for decisions
  - Arrows for flow of control
- Easier for non-technical audiences but less formal for complex interaction modeling.



JSD Diagrams (Jackson System Development)

- Focus on modeling the structure of processes in terms of input, output, and sequencing.
- Often used more in software engineering than directly in HCI, but still useful for complex system behavior descriptions.



In dialogue design, meaning (semantics) is added to actions and system states to make interaction clear and understandable. The dialogue can be managed centrally or divided across different parts of the system, but the goal is always to keep the structure clear. Good dialogue design ensures actions are complete, predictable, and consistent, while system states remain reachable, reversible, and safe. Visual aspects like visibility, layout, and style also affect how users experience the interaction.

9. Evaluation Techniques

Evaluation in HCI aims to check whether the system is usable and meets user needs. There are two main groups of evaluation methods:

1. Expert-Based Evaluation: In these methods, experts evaluate the system without involving real users.

Cognitive Walkthrough: Focuses on whether users can easily learn to use the system by reasoning through tasks. It Emphasizes user goals and required knowledge. For each task, the expert asks:

- What impact does this interaction have on the user?
- What cognitive processes does it require?
- What learning difficulties may occur?

What you need to prepare:

- System prototype or specification.
- Clear description of tasks.
- Full list of actions needed to perform each task.
- User profile description (who are the target users).

For each action, the expert answers:

1. Is the effect of the action aligned with the user's goal at that point?
2. Will users recognize that this action is available?
3. Once found, will users know it's the correct action?
4. After performing the action, will users understand the feedback?

Heuristic Evaluation: Experts evaluate the system based on a set of general usability principles (heuristics). Multiple evaluators perform the evaluation independently.
Works well in **early stages of design**.

Nielsen's 10 Heuristics:

1. **Visibility of system status:** keep users informed with timely, appropriate feedback.
2. **Match between system and real world:** speak the users' language, follow real-world conventions.
3. **User control and freedom:** allow easy recovery from mistakes (undo, cancel, exit).
4. **Consistency and standards:** follow platform conventions, avoid ambiguity.
5. **Error prevention:** design to avoid errors before they happen (e.g. confirmation before critical actions).

6. **Recognition rather than recall:** minimize memory load by making options and information visible.
7. **Aesthetic and minimalist design:** avoid unnecessary elements; only show relevant information.
8. **Help users recognize, diagnose, and recover from errors:** clear, helpful error messages with guidance.
9. **Help and documentation:** ideally not needed, but if present, it should be easy to find, search, and follow.

Evaluation through user participation: In these methods, real users are involved to directly observe their interaction with the system.

Think Aloud – Users perform tasks while verbalizing their thoughts:

- What they are doing.
- Why are they doing it?
- What they believe is happening.
- Provides valuable insights into users' reasoning, expectations, and difficulties.

Simple to perform, but results can be subjective and selective (depends on how well users can express themselves).

When to use:

- After developing a prototype.
- Tasks should reflect typical user goals.
- Participants should match the target personas.

After the session:

- Analyze the verbal data.
- Identify usability issues.
- Present and report findings.

Cooperative Evaluation – it's a variation of Think Aloud where the user is treated as a collaborator rather than just a participant. Users are encouraged to ask questions or request clarification when they face problems (in Think Aloud this is usually not allowed). This method helps uncover misunderstandings or confusions in real time.

Controlled Experiments – An experimental method where all conditions are kept the same except for one variable that is being tested – it tests the effects of a variable while keeping all other factors constant.

- ➔ The evaluator defines a **hypothesis** to test.
- ➔ Participants complete tasks under carefully controlled conditions.
- ➔ Only **one variable** is changed at a time.
- ➔ Changes in behavior are measured and attributed to that variable.

In controlled experiments, variables are:

- **independent** when they can be manipulated (e.g. interface style, number of menu items);
- **dependent** when they're the outcomes you measure (e.g. time, errors, user satisfaction);

Important aspects:

- Participants should match the intended user population.
- The experiment tests a hypothesis (prediction about expected results).
- The goal is to either confirm the hypothesis or disprove the null hypothesis (which assumes no difference between conditions).

Experimental designs:

- **Between-groups design:** Each participant is assigned to one condition (group), and comparisons are made between different groups (more users needed, no learning effect);

- **Within-groups design:** Each participant performs tasks under multiple conditions, allowing direct comparison within the same user (fewer participants needed, risk of learning effects);

Statistics play a key role in analyzing the results and determining whether observed differences are meaningful, or are obtained by chance. Through statistics we're able to accept or reject the hypothesis.

Hypothesis Testing: Hypothesis testing is a statistical method used to check if the difference between two or more conditions (e.g. designs, interfaces) is real or just due to random chance. In HCI, we often compare two versions of a system to see which one performs better.

- **Null Hypothesis (H_0):** assumes there is **no difference** between the conditions.
- **Alternative Hypothesis (H_1):** assumes there **is a difference** between the conditions.

We collect data by running experiments with users and analyze the results.

- If the data shows that the difference is unlikely to happen by chance (usually $p < 0.05$), we **reject H_0** and accept that a real difference exists.

ANOVA (Analysis of Variance): **ANOVA** is a statistical test used when you want to compare **more than two groups or conditions**.

- Instead of doing many pairwise comparisons, ANOVA checks if at least one condition is significantly different from the others.
- It helps avoid increasing error rates that happen when doing multiple individual tests.

How it works:

- It compares the variation **between groups** to the variation **within groups**.
- The result is an **F-value**:
  - If the F-value is large enough (greater than a critical value), it means that at least one group is different.
- If ANOVA finds a significant difference, further tests (post-hoc tests) can be used to identify exactly which groups differ.

Example (Icon Design Study) *Problem: Compare* ***natural vs. abstract icons*** *for memorability.*

- Hypothesis: Users will remember natural icons more easily.
- Independent variable: Icon style.
- Dependent variables: Number of errors, selection time.
- Method: Within-subjects design (participants tested both conditions).
- Randomize order to avoid learning effects.
- Measure time and errors for each user.

Sampling and Representation: The **sample** is the group tested; the **population** is the broader group you want to generalize to. Sampling should aim to match the real user population. Non-random sampling (e.g. snowball samples, biased user groups) can affect independence and validity.

sources of randomness & noise: how random is the world?

- **Noise:** uncontrolled variation in data (e.g. user mood, distractions).
- **Bias:** systematic errors that push results in one direction.
- **Independence:** is about whether one measurement or factor gives information about another; Independence can be influenced by **measurements**:
    - order effects: users see system A first, followed by system B
    - context or day effects: bad weather often affects people's moods
    - experimenter effects: contextual factor is you

  Or by **factor effects**: is when there is some form of relationship or correlation between the various factors that you are measuring aspects of;

  And lastly, **sample composition**:

  1. internal non-independence: is when subjects are likely to be similar to one another but in no particular direction with regard to you question – example: snowball sample;
  2. external non-independence: is when the choice of subjects is actually connected with the topic being studied – example: doing a survey about preferences between MacOs and windows in the apple store!

Experimental Design Considerations

- **Order effects:** learning from one condition can affect performance in the next.
- **Randomization:** helps reduce order effects.
- **Matched users:** pair similar users across conditions to reduce variability.
- **Targeted user groups:** select participants who will make effects more visible (e.g. older users for memory studies).

Task Design: as well as choosing *whom*, we can manipulate *what* we ask them to do

- **Distractor tasks:** overload some cognitive capacity to expose system differences (e.g. using an interface while walking).
- **Targeted tasks:** tasks that stress the system features you're testing.
- **Demonic interventions:** extreme tasks that deliberately highlight differences (less generalizable).
- **Restricted tasks:** tightly controlled tasks to minimize uncontrolled variables.